1-4

RAG 이론 & 실습

학습 목차

- **Retrieval Augmented Generation**
  - Fine-Tuning과의 비교
  - Naive -> Advanced -> Modular RAG 발전 과정
- **RAG 핵심 모듈**
  - Loading, Splitting, Embedding
  - Storage, Retrieval, Generation
- **청킹 전략과 임베딩**
  - Fixed-size, Recursive, Semantic Chunking
  - 임베딩 모델 선택 기준

© Scopelabs. All Rights Reserved

Retrieval Augmented Generation

Fine-Tuning과의 비교 (1)

RAG: 모델이 모르는 외부 지식을 검색해서 답변 생성에 함께 넣어주는 방식

- LLM은 학습 시점 이후의 정보나 사내 문서를 기본적으로 알 수 없음
- 질문과 관련 있는 문서를 먼저 찾고, 그 문서를 근거로 답변을 생성
- 모델을 다시 학습시키지 않아도 지식 기반을 바꿀 수 있음

```
당신은 사내 문서를 근거로 답변하는 AI Assistant입니다.
아래 맥락에 포함된 정보만 사용해 질문에 답하세요.
맥락에 답이 없으면 "문서에서 확인할 수 없습니다"라고 말하세요.

질문: {question}

맥락:
{context}
```

Fine-Tuning과의 비교 (2)

Fine-Tuning과 RAG는 해결하려는 문제가 다르다.

- Fine-Tuning: 모델의 말투, 출력 형식, 업무 스타일 조정에 유리
- RAG: 최신 정보, 사내 문서, 제품 매뉴얼 참조에 유리
- 실제 서비스에서는 기본 모델, 프롬프트, RAG, Fine-Tuning을 목적에 맞게 조합

> **Fine-Tuning은 모델을 학습시키고, RAG는 맥락을 추가한다.**

Naive -> Advanced -> Modular RAG 발전 과정 (1)

Naive RAG는 RAG의 가장 기본적인 형태

1. 문서를 불러온다.
2. 문서를 작은 청크로 나눈다.
3. 청크를 임베딩해서 저장한다.
4. 질문과 가까운 청크를 검색한다.
5. 검색 결과를 프롬프트에 넣고 답변을 생성한다.

Naive -> Advanced -> Modular RAG 발전 과정 (2)

Advanced RAG는 검색 품질과 답변 품질을 높이기 위한 보완 단계를 추가한 형태

- Query Rewrite: 사용자의 질문을 검색하기 좋은 형태로 다시 작성
- Hybrid Search: 키워드 검색과 벡터 검색을 함께 사용
- Re-ranking: 검색 결과의 우선순위를 다시 계산
- Self-check: 답변과 근거의 일치 여부를 점검

Naive -> Advanced -> Modular RAG 발전 과정 (3)

Modular RAG는 RAG 단계를 독립적인 모듈처럼 조합하는 방식

- Query Rewrite
- Retrieval
- Re-ranking
- Generation
- Self-check

RAG 핵심 모듈

Loading, Splitting, Embedding (1)

Loading: 원천 데이터를 RAG 시스템이 다룰 수 있는 문서 형태로 불러오는 단계

- PDF, 웹 페이지, 텍스트 파일, 사내 문서 등을 로드
- 원문 내용뿐 아니라 파일명, 페이지 번호, URL 같은 메타데이터도 중요
- 메타데이터가 있어야 답변에 출처를 붙이거나 문서를 필터링할 수 있음

```
from langchain_community.document_loaders import PDFMinerLoader

loader = PDFMinerLoader("./docs/manual.pdf")
documents = loader.load()

print(documents[0].page_content[:300])
print(documents[0].metadata)
```

Loading, Splitting, Embedding (2)

Splitting: 긴 문서를 검색하기 좋은 작은 단위로 나누는 단계

- 문서 전체를 한 번에 검색 대상으로 쓰기에는 너무 큼
- 너무 크게 나누면 필요한 부분만 찾기 어려움
- 너무 작게 나누면 의미가 끊겨 답변에 필요한 맥락이 부족해질 수 있음

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=120,
)
chunks = splitter.split_documents(documents)

print(len(chunks))
```

Loading, Splitting, Embedding (3)

Embedding: 텍스트의 의미를 벡터로 변환하는 단계

- 비슷한 의미의 문장은 벡터 공간에서도 가까운 위치에 놓이도록 변환
- 질문도 벡터로 바꿔서 가까운 문서 청크를 찾음
- RAG 검색 품질은 임베딩 모델 품질에 큰 영향을 받음

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small"
)

vector = embeddings.embed_query("RAG란 무엇인가?")
print(len(vector))
```

Storage, Retrieval, Generation (1)

Storage: 임베딩된 청크와 원문, 메타데이터를 저장하는 단계

- Chroma, FAISS 같은 벡터 저장소를 활용할 수 있음
- 벡터 검색을 빠르게 수행하기 위한 인덱스를 관리
- 문서 변경 시 기존 인덱스를 어떻게 갱신할지도 고려

```
from langchain_chroma import Chroma

vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db",
)
```

Storage, Retrieval, Generation (2)

Retrieval: 사용자의 질문과 관련 있는 문서 청크를 찾는 단계

- 질문을 임베딩한 뒤 벡터 저장소에서 가까운 청크 검색
- 검색 개수, 유사도 기준, 메타데이터 필터가 결과 품질에 영향
- 검색 결과가 틀리면 생성 단계가 좋아도 답변이 흔들릴 수 있음

```
retriever = vectorstore.as_retriever(
    search_kwargs={"k": 3}
)

docs = retriever.invoke("Agent에서 메모리는 왜 필요한가?")

for doc in docs:
    print(doc.metadata)
    print(doc.page_content[:200])
```

Storage, Retrieval, Generation (3)

Generation: 검색된 근거를 바탕으로 최종 답변을 생성하는 단계

- 모델에게 질문과 검색 결과를 함께 제공
- 검색된 근거 안에서 답변하도록 지시
- 필요한 경우 답변과 함께 출처 표시

청킹 전략과 임베딩

Fixed-size, Recursive, Semantic Chunking (1)

Fixed-size Chunking: 글자 수나 토큰 수 기준으로 일정하게 나누는 방식

- 구현이 단순하고 예측 가능
- 문장이나 문단 중간에서 잘릴 수 있음
- 문서 구조가 단순한 경우 빠르게 실험하기 좋음

Recursive Chunking: 문단, 문장, 구분자 순서로 자연스러운 경계를 찾는 방식

- 의미 단위가 비교적 잘 보존됨
- 일반적인 문서 처리에서 기본 선택지로 많이 사용
- 문서 구조가 어느 정도 정리되어 있을수록 효과적

Fixed-size, Recursive, Semantic Chunking (2)

Semantic Chunking: 의미 변화 지점을 기준으로 나누는 방식

- 주제가 바뀌는 지점을 기준으로 청크를 구성
- 보고서, 정책 문서처럼 의미 흐름이 중요한 자료에 적합
- 구현과 계산 비용은 상대적으로 더 커질 수 있음

임베딩 모델 선택 기준

임베딩 모델은 문장 의미를 벡터로 얼마나 잘 표현하는지를 결정

- 한국어 문서라면 한국어 의미 유사도를 잘 반영하는 모델이 필요
- 도메인 용어가 많으면 실제 데이터로 검색 품질을 확인해야 함
- 벡터 차원 수, 속도, 비용도 함께 고려

임베딩 모델 선택은 벤치마크 점수만으로 끝나지 않는다.

- 실제 사용자 질문 샘플
- 기대되는 정답 문서
- 청킹 전략별 검색 결과
- 비용과 응답 시간